- Snapshot, deploy, health check, rollback, fault injection
- `agent.py`, the terminal flow that is also the demo fallback
- Network and deploy tests

- Schema, injection, approval and audit tests
- Report, README, screenshots, backup video
- Reviews every pull request you open

Writing tests against your code is how your teammate learns the engine. Judges can ask either of you about any stage, so the teammate also does the manual routing exercise in phase 1.

Three things that block day 1

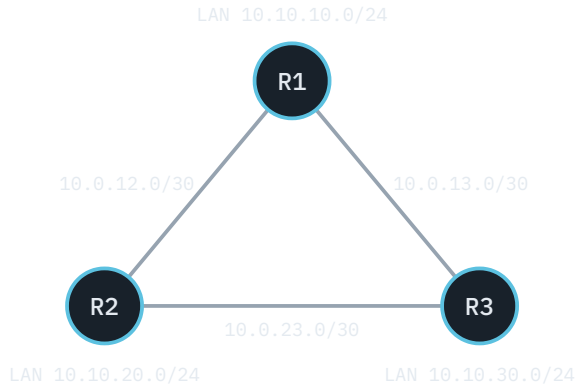
Your Mac cannot run Containerlab directly. It needs Linux. Install OrbStack, create a Linux machine, and run Docker and Containerlab inside it. Do this before anything else, because every later phase depends on the lab.

The mentor defines the 2 intents. The plan assumes ADD_ROUTE and REMOVE_ROUTE. Ask now. If the mentor wants OSPF, the twin design changes.

The report must follow `Proposed-structure-hackathon.pdf`. You do not have that file yet. Get

it from the AIORI site or the mentor so the report is written into the right headings from the start.

The lab you are building



A triangle, so a bad route can send traffic in a circle. That is what the twin has to catch.

- R1–R2: R1 is `10.0.12.1` , R2 is `10.0.12.2`
- R2–R3: R2 is `10.0.23.1` , R3 is `10.0.23.2`
- R1–R3: R1 is `10.0.13.1` , R3 is `10.0.13.2`

Where your two halves meet

Agree on this on day 3 and do not change it afterwards. The engine exposes three functions. The terminal and the API both call them, and nothing else touches the routers.

```
plan(intent: dict) -> Change        # schema, semantic, generate, twin, report
apply(change_id, approver, reason,  # backup, deploy, health check,
      simulate_failure=False)       #   rollback on failure
reject(change_id, approver, reason)

Change.status: INVALID | BLOCKED | PENDING_APPROVAL | REJECTED
             | SUCCESS | ROLLED_BACK | ROLLBACK_FAILED
```

Your teammate can build the API and dashboard against a fake `plan()` that returns a hard-coded Change, then swap in the real one in phase 5.

Run sheet

Everyone | You | Mate

0 / 56

Progress is saved with this page and shared with anyone you give Contributor access.

DAY 1 Unblock

0 / 7

- [ ] YOU — Install OrbStack, create a Linux machine, install Docker and Containerlab inside it
  Run the Containerlab quick-start lab once to prove it works.
- [ ] MATE — Create the private GitHub repo named after the team; add you and `aiori-hackathon` as collaborators

Transfer ownership to aiori-hackathon once they accept.

- [ ] MATE — Ask the mentor to confirm ADD_ROUTE and REMOVE_ROUTE as the 2 intents
- [ ] MATE — Get `Proposed-structure-hackathon.pdf` and put it in `docs/`
- [ ] MATE — Same Linux VM, Docker and Containerlab setup on the teammate's laptop
- [ ] YOU — Scaffold the repo: `lab/` , `backend/` , `tests/` , `docs/` , `snapshots/` , `agent.py` , `requirements.txt`
- [ ] BOTH — Agree the working rule: feature branches, the other person reviews every pull request

DONE WHEN Both laptops run `containerlab version` and both of you can push to the repo.

DAYS 1–3 Learn the network by hand

0 / 6

- [ ] BOTH — Networking basics: CIDR, /30 and /24, routing table, next-hop, loop, blackhole, traceroute
  Practical Networking on YouTube covers all of it.
- [ ] YOU — Write `lab/topology.clab.yml` and a base `frr.conf` per router from the IP plan
- [ ] YOU — Write `lab/inventory.yaml` : routers, link IPs, LANs
  Every later module reads this. No IPs hard-coded in Python.
- [ ] YOU — In vtysh, add static routes by hand on all 3 routers and ping every LAN
- [ ] BOTH — Create a routing loop by hand, watch traceroute bounce, then remove it
  Each of you does this on your own laptop. It is the bug your product exists to stop.
- [ ] MATE — Record every vtysh command used in `docs/lab-notes.md`
  This becomes the topology section of the report.

DONE WHEN `containerlab deploy` starts 3 routers and every LAN pings every other LAN, on both laptops.

DAYS 3–5 Intent to command

0 / 8

- [ ] BOTH — Freeze the contract: intent JSON, the three engine functions, the Change record
- [ ] YOU — Pydantic models for ADD_ROUTE and REMOVE_ROUTE
  A router name given as next_hop is converted to the link IP before validation.
- [ ] YOU — Read live static routes from a router with `docker exec` and vtysh
  Semantic checks need it now; the twin reuses it.

- [ ] YOU Semantic checks: next-hop directly connected, conflict, duplicate, route exists for REMOVE
- [ ] YOU Jinja2 templates plus allowlist: every output line starts with `ip route` or `no ip route`
  REMOVE looks up the existing next-hop first. CIDR everywhere.
- [ ] MATE `audit/db.py` : SQLite change record with create, update stage, get, list
- [ ] MATE Tests 3, 4, 5, 16 and 13 against your models
- [ ] MATE Export the JSON Schema from Pydantic into `docs/`

DONE WHEN A good intent prints the right FRR command, bad ones print a clear reason, and the teammate's 5 tests pass.

DAYS 5–8 Digital twin

0 / 10

- [ ] YOU Network model built from the inventory plus live routes
- [ ] YOU Apply the candidate to a copy of the model, never the original
- [ ] YOU Loop check: for each destination, walk next-hops from every router; a repeat visit is a loop
- [ ] YOU Blackhole and reachability checks
  LANs that reached each other before must still reach each other after.
- [ ] YOU Syntax check: run the candidate through FRR's own parser inside a router container
- [ ] YOU Validation report: one PASS or FAIL line per check, then SAFE TO REVIEW or the reason
- [ ] YOU Tests 6, 7, 8, 14, 15, 17
- [ ] MATE FastAPI: POST `/intent` , `/validate` , `/approve` , `/reject` ; GET `/routers` , `/changes` , `/changes/{id}`
  Built against a fake plan() until phase 5.
- [ ] MATE Approval logic with a 5-minute timeout that counts as reject; tests 9 and 10
  Make the timeout a parameter so the test can use 1 second.
- [ ] MATE Report: problem, proposed solution, architecture diagram, topology and IP plan

DONE WHEN An intent that would loop is blocked, and the message names the path R1 → R2 → R3 → R1.

DAYS 8–10 Deploy and rollback

0 / 8

☐	YOU	Snapshot all 3 running-configs to `snapshots/<change_id>/` and git commit
☐	YOU	Apply with vtysh as one transaction: any router failing fails the whole change
☐	YOU	Health check: route present or absent as expected, LAN pings pass
☐	YOU	Rollback: restore the snapshot on all 3 routers, then run the health check again Mark ROLLED_BACK, or ROLLBACK_FAILED and alert the operator.
☐	YOU	`--simulate-failure` flag that breaks the deploy on purpose
☐	YOU	Tests 11, 12, 19
☐	MATE	Dashboard screens 1 and 2: Network Status, Create Change
☐	MATE	Report: intent schema, config generation, digital twin and validation

DONE WHEN A forced failure restores the network on its own, and each running-config equals its snapshot.

DAYS 10–11 Integration

0 / 4

☐	YOU	`python agent.py apply intent.json` with a terminal approve or reject prompt
☐	BOTH	Swap the fake plan() for the real engine in the API and run the full flow through it
☐	MATE	Tests 1, 2, 18, 20
☐	BOTH	Run the four demo scenarios back to back and write down anything that felt slow or unclear

DONE WHEN All four demo runs work from the terminal: good change, loop blocked, human reject, failed deploy rolled back. Everything after this is polish.

DAYS 11–14 Dashboard and test report

0 / 6

☐	MATE	Dashboard screens 3 to 5: Validation Result, Approval with red and green diff, History
☐	BOTH	Day 12 check: if the dashboard cannot show one real change end to end, switch to Streamlit
☐	YOU	Run all tests, fix failures, save the report to `docs/test-results`
☐	YOU	Tighten error messages so every stop states which stage and why
☐	MATE	`[illegible]` that starts backend and frontend together

☐	MATE	`docker-compose.yml` that starts backend and frontend together
☐	MATE	Optional: LLM turns plain English into intent JSON, which then goes through the normal pipeline Only if every gate above is green.

DONE WHEN `pytest` shows 20 passing, or at least the 12 must-haves, and the result is saved in `docs/test-results`.

DAYS 14–16 Report, demo, submit 0 / 7

☐	MATE	Finish the report PDF in AIORI's structure, with the 9 security principles mapped to code files
☐	YOU	Review the report's technical sections and write limitations and future scope NETCONF (RFC 6241) and YANG (RFC 7950) go here.
☐	YOU	Pseudocode snippet for the mentor
☐	MATE	README with setup steps and screenshots
☐	BOTH	Rehearse the 6-minute demo three times and record a backup video
☐	BOTH	Quiz each other on every pipeline stage until either of you can answer for any part
☐	MATE	Upload the report PDF and confirm the repo transfer to `aiori-hackathon`

DONE WHEN Report PDF is in the repo, the mentor has the pseudocode, the backup video exists, and ownership is transferred.

If you fall behind, cut in this order

1. LLM front door. The PS says it is optional.
2. Second Containerlab copy as a container twin. The Python model already proves loops.
3. Hash chain on the audit log.
4. Next.js. Switch to Streamlit, or demo from `agent.py`.
5. Tests 13–20. The PS asks for 12.

Never cut: the approval gate, automatic rollback, the twin's loop check, 12 passing tests across all four PS categories, the report, and the repo transfer. Those are what the PS grades and what the certificate depends on.

Test ownership